Object Oriented Programming: Python BootCamp

HIT-emlyon BSc Program
April-May, Harbin | © Imène BRIGUI | Saeed VARASTEH

Lecture 07 : Visualization

Data visualization is a broad topic, complete with its own theory and a host of tools for displaying and interacting with visualizations. In this section, you will get an introduction to the **matplotlib** package, which is one of the more popular packages for quickly creating two-dimensional figures.

Recap: You learned how to work with arrays of data using the NumPy package. While NumPy makes working with and manipulating data simple, it does not provide a means for human consumption of data. For that, you need to **visualize** your data.

```
import matplotlib.pyplot as plt
import numpy as np
```

The first example

```
my_list = [10, 20, 25, 35]
plt.plot(my_list);
```

Anatomy of a "Plot"

centered image

The `Figure` is the top-level container in this hierarchy. It is the overall window/page that everything is drawn on. You can have multiple independent figures and `Figure` s can contain multiple `Axes` / `Subplots` .

Each `Axes` has an `XAxis` and a `YAxis` . These contain the ticks, tick locations, labels, etc. In this tutorial, we'll mostly control ticks, tick labels, and data limits through other mechanisms, so we won't touch the individual `Axis` part of things all that much. However, it is worth mentioning here to explain where the term `Axes` comes from.

Figure

To manage multiple plots and subplots it makes sense to explcitly create the `figure` and `axes` objects.

```
fig, axes = plt.subplots(nrows=1, ncols=1)
```

`plt.subplots` returns us both a figure and an array of axes/subplots.

If we want to created multiple subplots:

```
fig, axes = plt.subplots(nrows = 2, ncols = 4)
```

improving appearance with `fig.tight_layout()` and `figsize`

Right now the figure is very small and also the axis tick labels are overlapping.
To fix that we can call `fig.tight_layout()`

```
fig, axes = plt.subplots(nrows = 2, ncols = 4)
fig.tight_layout()
```

And also change change the figure size with the parameter `figsize`

```
fig, axes = plt.subplots(nrows = 2, ncols = 4, figsize=(10, 4))
fig.tight_layout()
```

`set` ting up our subplot

```
fig, ax = plt.subplots(nrows = 1, ncols = 1)
```

Matplotlib's objects typically have lots of "explicit setters" -- in other words, functions that start with set_<something> and control a particular option.

To demonstrate this (and as an example of IPython's tab-completion), try typing `ax.set_` in a code cell, then hit the <Tab> key. You'll see a long list of `Axes` methods that start with `set` .

So lets `set` up some things

```
ax.set_xlim([0.5, 4.5])
ax.set_ylim([-2, 8])
ax.set_title("An Example Axes")
ax.set_ylabel("Y-Axis")
ax.set_xlabel("X-Axis")
```

In notebooks we can show our figure just by writting its variable

```
fig
```

Plotting Functions matplotlib plots

conditional bar plots

```
np.random.seed(1)

x = np.arange(5)
y = np.random.randn(5)

fig, ax = plt.subplots()

vert_bars = ax.bar(x, y)

for bar, height in zip(vert_bars, y):
    if height < 0:
        bar.set(color='red')

ax.axhline(y=0, color='black', linewidth=2);
```

Displaying 2d data with `imshow`

with `imshow` we can also display images

```
fig, ax = plt.subplots()

img = plt.imread("images/cat.jpg")
ax.imshow(img)
ax.axis("off");
```

Scatter for n-dimensional data

`scatter` allows to map several dimensions to different aesthetics such as x-postion, color, size and shape.

```
n = 50
x1 = np.random.random(n)
x2 = np.random.random(n) * 50
x3 = np.random.random(n)


y = x1 + x2 + x3

fig, ax = plt.subplots()
#https://matplotlib.org/3.5.0/api/_as_gen/matplotlib.axes.Axes.scatter.html
sc = ax.scatter(x=x1, y=y, s=x2, c=x3, marker='o')
fig.colorbar(sc)
#plt.show()
```

EXERCISES 05 - PART TWO

SELF-STUDY

The "language" of Matplotlib.

This section will go over many of the properties that are used throughout this library.

Colors

This is, perhaps, the most important piece of vocabulary in Matplotlib. Given that Matplotlib is a plotting library, colors are associated with everything that is plotted in your figures. Matplotlib supports a very robust language for specifying colors that should be familiar to a wide variety of users.

By default matplotlib will choose different colors when combining data on the same axes.

```
t = np.arange(0.0, 5.0, 0.2)
fig, ax = plt.subplots()
ax.plot(t, t, linewidth=5)
ax.plot(t, t**2, linewidth=5)
ax.plot(t, t**3, linewidth=5)
plt.show()
```

Colornames

Fi#rst, colors can be given as strings. For very basic colors, you can even get away with just a single letter:

- b: blue
- g: green
- r: red
- c: cyan
- m: magenta
- y: yellow
- k: black
- w: white

Other colornames that are allowed are the HTML/CSS colornames such as "burlywood" and "chartreuse". See the full list of the 147 colornames.

```
t = np.arange(0.0, 5.0, 0.2)
fig, ax = plt.subplots()
ax.plot(t, t, linewidth=5, color="r")
ax.plot(t, t**2, linewidth=5, color="blue")
ax.plot(t, t**3, linewidth=5, color="yellowgreen")
plt.show()
```

Hex values

Colors can also be specified by supplying a HTML/CSS hex string, such as `'#0000FF'` for blue. Support for an optional alpha channel was added for v2.0. For more information about hex colors have a look at https://en.wikipedia.org/wiki/Web_colors#Hex_triplet.

```
t = np.arange(0.0, 5.0, 0.2)
fig, ax = plt.subplots()
ax.plot(t, t, linewidth=5, color='#00ffff')
ax.plot(t, t**2, linewidth=5, color='#ff00ff')
ax.plot(t, t**3, linewidth=5, color='#ffcc00')
plt.show()
```

RGB[A] tuples

You may come upon instances where the previous ways of specifying colors do not work. This can sometimes happen in some of the deeper, stranger levels of the library. When all else fails, the universal language of colors for matplotlib is the RGB[A] tuple. This is the "Red", "Green", "Blue", and sometimes "Alpha" tuple of floats in the range of [0, 1]. One means full saturation of that channel, so a red RGBA tuple would be `(1.0, 0.0, 0.0, 1.0)` , whereas a partly transparent green RGBA tuple would be `(0.0, 1.0, 0.0, 0.75)` . The documentation will usually specify whether it accepts RGB or RGBA tuples. Sometimes, a list of tuples would be required for multiple colors, and you can even supply a Nx3 or Nx4 numpy array in such cases.

In functions such as `plot()` and `scatter()` , while it may appear that they can take a color specification, what they really need is a "format specification", which includes color as part of the format. Unfortunately, such specifications are string only and so RGB[A] tuples are not supported for such arguments (but you can still pass an RGB[A] tuple for a "color" argument).

Oftentimes there is a separate argument for "alpha" where-ever you can specify a color. The value for "alpha" will usually take precedence over the alpha value in the RGBA tuple. There is no easy way around this inconsistency.

```
t = np.arange(0.0, 3.0, 0.2)
fig, ax = plt.subplots()
ax.plot(t, t, linewidth=5, color=(0, 0, 1))
ax.plot(t, t**2, linewidth=5, color=(0, 0, 1, 0.5))
# the alpha value can also be specified as an additional kwarg
ax.plot(t, t**3, linewidth=5, color="b", alpha=0.5)
plt.show()
```

256 Shades of Gray

A gray level can be given instead of a color by passing a string representation of a number between 0 and 1 (inclusive). `'0.0'` is black, while `'1.0'` is white. `'0.75'` would be a light shade of gray.

```
t = np.arange(0.0, 5.0, 0.2)
fig, ax = plt.subplots()
ax.plot(t, t, linewidth=5, color='1.0')
ax.plot(t, t**2, linewidth=5, color='0.5')
ax.plot(t, t**3, linewidth=5, color='0.0')
plt.show()
```

Cycle references

With the advent of fancier color cycles coming from the many available styles, users needed a way to reference those colors in the style without explicitly knowing what they are. So, in v2.0, the ability to reference the first 10 iterations of the color cycle was added. Whereever one could specify a color, you can supply a 2 character string of 'C#'. So, 'C0' would be the first color, 'C1' would be the second, and so on and so forth up to 'C9'.

```
t = np.arange(0.0, 5.0, 0.2)
fig, ax = plt.subplots()
ax.plot(t, t, linewidth=5)
ax.plot(t, t**2, linewidth=5)
ax.plot(t, t**3, linewidth=5)
plt.show()
```

```
t = np.arange(0.0, 5.0, 0.2)
fig, ax = plt.subplots()
ax.plot(t, t, linewidth=5, color='C0')
ax.plot(t, t**2, linewidth=5, color='C1')
ax.plot(t, t**3, linewidth=5, color='C2')
plt.show()
```

Markers

Markers are commonly used in `plot()` and `scatter()` plots, but also show up elsewhere. There is a wide set of markers available, and custom markers can even be specified.

marker	description	marker	description	marker	description	marker	description
"."	point	"+"	plus	","	pixel	"x"	cross
"o"	circle	"D"	diamond	"d"	thin_diamond		
"8"	octagon	"s"	square	"p"	pentagon	"*"	star
"|"	vertical line	"_"	horizontal line	"h"	hexagon1	"H"	hexagon2
0	tickleft	4	caretleft	"<"	triangle_left	"3"	tri_left
1	tickright	5	caretright	">"	triangle_right	"4"	tri_right
2	tickup	6	caretup	"^"	triangle_up	"2"	tri_up
3	tickdown	7	caretdown	"v"	triangle_down	"1"	tri_down
"None"	nothing	`None`	default	" "	nothing	""	nothing

```
t = np.arange(0.0, 5.0, 0.2)
fig, ax = plt.subplots()
ax.plot(t, t, '.', linewidth=5)
ax.plot(t, t**2, 'o', linewidth=5)
ax.plot(t, t**3, marker='+', linewidth=1) # With explicit arguments, you can set maker and linestyle separately.
ax.plot(t, -t, ls='', marker='v', linewidth=5)
plt.show()
```

Linestyles

Line styles are about as commonly used as colors. There are a few predefined linestyles available to use. Note that there are some advanced techniques to specify some custom line styles. Here is an example of a custom dash pattern.

linestyle	description
'-'	solid
'--'	dashed
'-.'	dashdot
':'	dotted
'None'	draw nothing
' '	draw nothing
''	draw nothing

Also, don't mix up ".-" (line with dot markers) and "-." (dash-dot line) when using the `plot` function!

```
t = np.arange(0.0, 5.0, 0.2)
fig, ax = plt.subplots()
ax.plot(t, t, linestyle='-', linewidth=5)
ax.plot(t, t**2, linestyle='--', linewidth=5)
ax.plot(t, t**3, linestyle='-.', linewidth=5)
ax.plot(t, -t, linestyle=':', linewidth=5)
plt.show()
```

```
fig, ax = plt.subplots(1, 1)
ax.bar([1, 2, 3, 4], [10, 20, 15, 13], linestyle='--', edgecolor='r', linewidth=4)
plt.show()
```

Colormaps

Another very important property of many figures is the colormap. The job of a colormap is to relate a scalar value to a color. In addition to the regular portion of the colormap, an "over", "under" and "bad" color can be optionally defined as well. NaNs will trigger the "bad" part of the colormap.

As we all know, we create figures in order to convey information visually to our readers. There is much care and consideration that have gone into the design of these colormaps. Your choice in which colormap to use depends on what you are displaying. In mpl, the "jet" colormap has historically been used by default, but it will often not be the colormap you would want to use. Much discussion has taken place on the mailing lists with regards to what colormap should be default. The v2.0 release of Matplotlib adopted a new default colormap, 'viridis', along with some other stylistic changes to the defaults.

```
example_x = np.random.normal(size=500)
example_y = np.random.normal(size=500)
example_z = example_x + example_y

plt.scatter(example_x, example_y, c=example_z, cmap="jet")
plt.show()
```

```
plt.scatter(example_x, example_y, c=example_z, cmap="viridis")
plt.show()
```

```
def plot_cmap(name, value_range=(0, 1)):
    gradient = np.linspace(*value_range, 256)
    gradient = np.vstack((gradient, gradient))
    fig, ax = plt.subplots(figsize=plt.figaspect(0.1))
    ax.imshow(gradient, aspect='auto', cmap=plt.get_cmap(name), vmin=0, vmax=1)
    pos = list(ax.get_position().bounds)
    x_text = pos[0] - 0.01
    y_text = pos[1] + pos[3]/2.
    ax.set_title(name, fontsize=20)
    ax.axis("off")

plot_cmap("jet")
```

```
plot_cmap("viridis")
```

Here you can find the full gallery of all the pre-defined colormaps, organized by the types of data they are usually used for: https://matplotlib.org/3.1.0/tutorials/colors/colormaps.html

Mathtext

Oftentimes, you just simply need that superscript or some other math text in your labels. Matplotlib provides a very easy way to do this for those familiar with LaTeX. Any text that is surrounded by dollar signs will be treated as "mathtext". Do note that because backslashes are prevelent in LaTeX, it is often a good idea to prepend an `r` to your string literal so that Python will not treat the backslashes as escape characters.

```
print(r"a\nb")
```

```
fig, ax = plt.subplots()
ax.scatter([1, 2, 3, 4], [4, 3, 2, 1])
ax.spines['top'].set(visible=False)
# Removing spines so they don't intersect with the title. tight_layout() is not sufficient here.
ax.spines['right'].set(visible=False)
ax.set_title(r'$\sigma_i=\frac{3}{5}$', fontsize=25)
plt.show()
```

Axis Decoration

In this section, we'll focus on what happens around the edges of the axes: Ticks, ticklabels, limits, layouts, and legends.

Legends

As you've seen in some of the examples so far, the X and Y axis can also be labeled, as well as the subplot itself via the title.

However, another thing you can label is the line/point/bar/etc that you plot. You can provide a label to your plot, which allows your legend to automatically build itself.

```
fig, ax = plt.subplots()
ax.plot([1, 2, 3, 4], [10, 20, 25, 30])  # Paris
ax.plot([1, 2, 3, 4], [30, 23, 13, 4])  # Lyon
ax.set(ylabel='Temperature (deg C)', xlabel='Time', title='A tale of two cities')
plt.show()
```

```
fig, ax = plt.subplots()
ax.plot([1, 2, 3, 4], [10, 20, 25, 30], label='Paris')
ax.plot([1, 2, 3, 4], [30, 23, 13, 4], label='Lyon')
ax.set(ylabel='Temperature (deg C)', xlabel='Time', title='A tale of two cities')
ax.legend()
plt.show()
```

The keyword argument `loc` allows to position the legend at different positions. The `'best'` argument is the default one which automatically chooses the location which overlaps the plot elements as little as possbile.

Location String	Location Code
best	0
upper right	1
upper left	2
lower left	3
lower right	4
right	5
center left	6
center right	7
lower center	8
upper center	9
center	10

```
fig, ax = plt.subplots()
ax.plot([1, 2, 3, 4], [10, 20, 25, 30], label='Philadelphia')
ax.plot([1, 2, 3, 4], [30, 23, 13, 4], label='Boston')
ax.set(ylabel='Temperature (deg C)', xlabel='Time', title='A tale of two cities')
ax.legend(loc='center')
plt.show()
```

Ticks, Tick Lines, Tick Labels and Tickers

This is a constant source of confusion:

- A Tick is the *location* of a Tick Label.
- A Tick Line is the line that denotes the location of the tick.
- A Tick Label is the text that is displayed at that tick.
- A `Ticker` automatically determines the ticks for an Axis and formats the tick labels.

`tick_params()` is often used to help configure your tickers.

```
fig, ax = plt.subplots()
ax.plot([1, 2, 3, 4], [10, 20, 25, 35])

# Manually set ticks and tick labels *on the x-axis* (note ax.xaxis.set, not ax.set!)
ax.xaxis.set(ticks=range(1, 5), ticklabels=[3, 100, -12, "foo"])

# Make the y-tick lines a bit longer and go in...
ax.tick_params(axis='y', direction='in', length=10)

plt.show()
```

```
data = [('apples', 2), ('oranges', 3), ('peaches', 1)]
fruit, value = zip(*data)

fig, ax = plt.subplots()
x = np.arange(len(fruit))
ax.bar(x, value, align='center', color='gray')
ax.set(xticks=x, xticklabels=fruit)
plt.show()
```

Subplot Spacing

The spacing between the subplots can be adjusted using `fig.subplots_adjust()` . Play around with the example below to see how the different arguments affect the spacing.

```
fig, axes = plt.subplots(2, 2, figsize=(9, 9))
fig.subplots_adjust(wspace=0.3, hspace=-0.7,
                    left=0.125, right=0.8,
                    top=0.7,    bottom=0.2)
plt.show()
```

A common "gotcha" is that the labels are not automatically adjusted to avoid overlapping those of another subplot. Matplotlib does not currently have any sort of robust layout engine, as it is a design decision to minimize the amount of "magical plotting". We intend to let users have complete, 100% control over their plots. LaTeX users would be quite familiar with the amount of frustration that can occur with automatic placement of figures in their documents.

That said, there have been some efforts to develop tools that users can use to help address the most common compaints. The "Tight Layout" feature, when invoked, will attempt to resize margins and subplots so that nothing overlaps.

If you have multiple subplots, and want to avoid overlapping titles/axis labels/etc, `fig.tight_layout` is a great way to do so:

```
def example_plot(ax):
    ax.plot([1, 2])
    ax.set_xlabel('x-label', fontsize=16)
    ax.set_ylabel('y-label', fontsize=8)
    ax.set_title('Title', fontsize=24)
```

```
fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(nrows=2, ncols=2)
example_plot(ax1)
example_plot(ax2)
example_plot(ax3)
example_plot(ax4)

plt.show()
```

```
fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(nrows=2, ncols=2)
example_plot(ax1)
example_plot(ax2)
example_plot(ax3)
example_plot(ax4)

# Tight layout enabled.
fig.tight_layout()

plt.show()
```
